Mandelbrot - HPC

Nicola Sebastianutto

Anno Accademico 2025/2026

Contents

1 Technical Cluster Specifications (Hardware & Software)

This section describes the hardware and software configurations of the two computing clusters utilized for the benchmarking: **Orfeo** and **Leonardo**.

1.1 Hardware Identification

The physical characteristics of the compute nodes on both systems were identified using `lscpu` and `numactl -H`. Table 1 summarizes the key hardware specifications of the allocated resources on both clusters.

Table 1: Hardware specifications of the computing nodes on Orfeo and Leonardo.

Feature	**Orfeo (GENOA Partition)**	**Leonardo (`dcgp_usr_prod`)**
Processor Model	AMD EPYC 9374F 32-Core	Intel Xeon Platinum 8480+
Sockets per Node	2	2
Cores per Socket	32	56
Total Cores per Node	64	112
NUMA Configuration	8 domains (8 cores/domain)	4 domains (SNC-4 mode, 28 cores/domain)
System Memory	512 GB (64 GB per NUMA node)	512 GB (514000 MB total)
L1d Cache	2 MiB (64 instances)	5.25 MiB (112 instances)
L1i Cache	2 MiB (64 instances)	3.5 MiB (112 instances)
L2 Cache	64 MiB (64 instances)	224 MiB (112 instances)
L3 Cache	512 MiB total (16 instances)	210 MiB total (2 instances)

In **Orfeo**, the Genoa partition utilizes AMD EPYC 9374F processors distributed across 8 NUMA domains per node, with 8 physical cores allocated to each domain.

In **Leonardo**, the partition (`dcgp_usr_prod`) utilizes the Intel Sapphire Rapids processors, that has 112 physical cores per node across two sockets.

The software environment, including compiler versions and MPI libraries, was inspected with `gcc --version` and `mpirun --version`.

Table 2: Software stack versions on Leonardo and Orfeo clusters.

Cluster	**GCC Compiler Version**	**MPI Library Version**
Leonardo	12.2.0 (Spack GCC)	Open MPI 4.1.6
Orfeo	14.3.1 (Red Hat)	Intel MPI 2021.17

Compilation on both systems was performed with optimization flags tailored to the target architectures. The build process was governed by the following base flags:

```
CFLAGS = -std=c11 -O3 -march=$(MARCH) -ffast-math -Wall -Wextra -pedantic
```

The architecture-specific micro-architecture flags (`-march`) were determined dynamically via compiler autodetect strategies:

- **Orfeo**: Compilations targeted AMD Family 19h core based CPUs architecture using `-march=znver4` (or falling back to modern SIMD extensions if needed).
- **Leonardo**: Compilations targeted the Intel sapphirerapids CPU with 64-bit extensions architecture using `-march=sapphirerapids`.

1.2 Run Configurations & Thread Pinning

To support the consistency and reproducibility of the benchmark results, thread pinning was managed via OpenMP environment variables. This approach minimizes cache misses and prevents performance degradation caused by arbitrary operating system thread migrations across socket and NUMA boundaries.

The runtime configurations were controlled using the following environment variables:

```
export OMP_NUM_THREADS=<num>
export OMP_PLACES=cores
export OMP_PROC_BIND=close
```

The rationale for this configuration is structured as follows:

- `OMP_NUM_THREADS`: Synchronizes the number of OpenMP execution threads with the exact CPU resources allocated per Slurm task, preventing thread oversubscription.
- `OMP_PLACES=cores`: Binds each OpenMP thread directly to a single physical core, ensuring that execution is not split across logical hyper-threads.
- `OMP_PROC_BIND=close`: Directs the runtime to place consecutive threads close to the master thread. This strategy keeps threads grouped within the same NUMA domain or socket, maximizing L1/L2 cache locality and reducing inter-socket memory latency.

2 Verification of Code Correctness

It is important to mathematically verify the correctness of the parallel implementation against the sequential baseline before conducting performance and scalability benchmarks.

2.1 Commutative Checksum Design

In sequential execution, the grid pixels are processed in a deterministic, linear order. This allows the use of a standard sequential checksum to verify data integrity. However, in parallel executions, especially with the dynamic master-worker paradigm used in the Mariani-Silveri algorithm where the computational domain is split into tiles. Due to asynchronous scheduling, load imbalance, and dynamic communications, these tiles are completed and returned to the master process in an unpredictable way and non-deterministic order.

To verify code correctness without degrading performance synchronization or sorting step, it is implemented a commutative and associative checksum. The checksum is represented by a structure containing four distinct slots:

```
typedef struct {
    unsigned int slot[4];
} checksum_t;
```

During the rendering of each pixel, the local checksum is updated using the `my_checksum_update` function:

```
checksum_t my_checksum_update(checksum_t checksum,
                              unsigned int value1,
                              unsigned int value2,
                              unsigned int value3) {
    unsigned int idx = (value1 + value3) % 4;
    unsigned int val = value2 + value3;

    checksum.slot[idx] += val;
    return checksum;
}
```

Here, `value1` and `value3` correspond to spatial coordinate metrics of the processed pixel, while `value2` represents the iteration count. Because the update operation it is based only by integer addition (which is both associative and commutative), the final value accumulated in each of the four slots is independent of the order in which individual pixels or whole tiles are processed.

At the end of the execution, each worker transmits its localized statistics block (including its local `checksum_t`) to the Master. The Master then accumulates these metrics using the `aggregateStats` function:

```
void aggregateStats(render_stats_t *master_stats, render_stats_t worker_stats) {
    for (unsigned int i = 0; i < 4; i++) {
        master_stats->my_checksum.slot[i] += worker_stats.my_checksum.slot[i];
    }
    master_stats->total_iterations += worker_stats.total_iterations;
    master_stats->inside_pixels    += worker_stats.inside_pixels;
    master_stats->time_waiting     += worker_stats.time_waiting;
    master_stats->time_working     += worker_stats.time_working;
    master_stats->tiles_processed  += worker_stats.tiles_processed;
}
```

By summing the array slots element, the Master computes a global checksum. This value has been verified to be identical to the one produced by the sequential implementation, confirming that the parallel execution yields mathematically consistent pixel results.

2.2 Validation under Edge Case Configurations

In addition to standard operational scenarios, edge-case tests were run to evaluate the stability of the master-worker coordination logic. One such verification test was executed with parameters designed to limit the number of available tasks below the number of active cpus. For example:

```
mpirun -n 4 ./build/mandel_mariani_mpi --brute 4096 --ppu 1 --dx-factor 2 --dy-factor 1
```

With these parameters, the domain decomposition factors (`--dx-factor 2` and `--dy-factor 1`) enforce a division of the computational space into exactly two tiles.

The outputs obtained from these edge-case benchmarks confirm that the communication, task distribution, and termination sequences function correctly under restrictive parallel configurations.

2.3 Mismatch Analysis: Brute-Force vs. Mariani-Silveri

To verify the accuracy of our parallel implementations, we analyzed the discrepancies between the baseline brute-force algorithm and the optimized Mariani-Silveri algorithm.

While the brute-force approach evaluates the complex iteration sequence for every single pixel in the domain, the Mariani-Silveri algorithm implements an optimization heuristic based on block boundaries. If the perimeter of a sub-grid block consists entirely of pixels that fall inside the Mandelbrot set, the algorithm classifies the internal of that block as pixels of the Mandelbrot Set.

This heuristic introduces a substantial performance speedup but can theoretically have approximation discrepancies since it may exist filamentary boundary regions, where thin structures may exit and re-enter a block's boundaries in an undetected way.

To quantify these differences, we computed the percentage error of the interior pixel count relative to the total pixel grid size across all combinations of resolution and maximum iteration limits ($k_{\max}$):

$$\text{Percentage Error } (\%) = \frac{|N_{\text{inside, brute}} - N_{\text{inside, Mariani}}|}{N_{\text{total}}} \times 100 \tag{1}$$

As shown in the heatmap, at lower resolutions or low $k_{\max}$ thresholds, there is complete agreement (0% error) between the two methods. This confirms that the optimization heuristic successfully preserves computational accuracy under standard rendering conditions.

At higher resolutions (such as 16384×16384 and 32768×32768) and higher iteration bounds ($k_{\max} \geq 128$), the boundary structures of the Mandelbrot set resolve into increasingly intricate geometries. Under these conditions, the boundary-checking heuristic introduces discrepancies. Although, at the maximum tested resolution and iteration limit, the error remains very small, demonstrating that the computational speedup achieved by the Mariani-Silveri optimization does not compromise the overall correctness of the output.

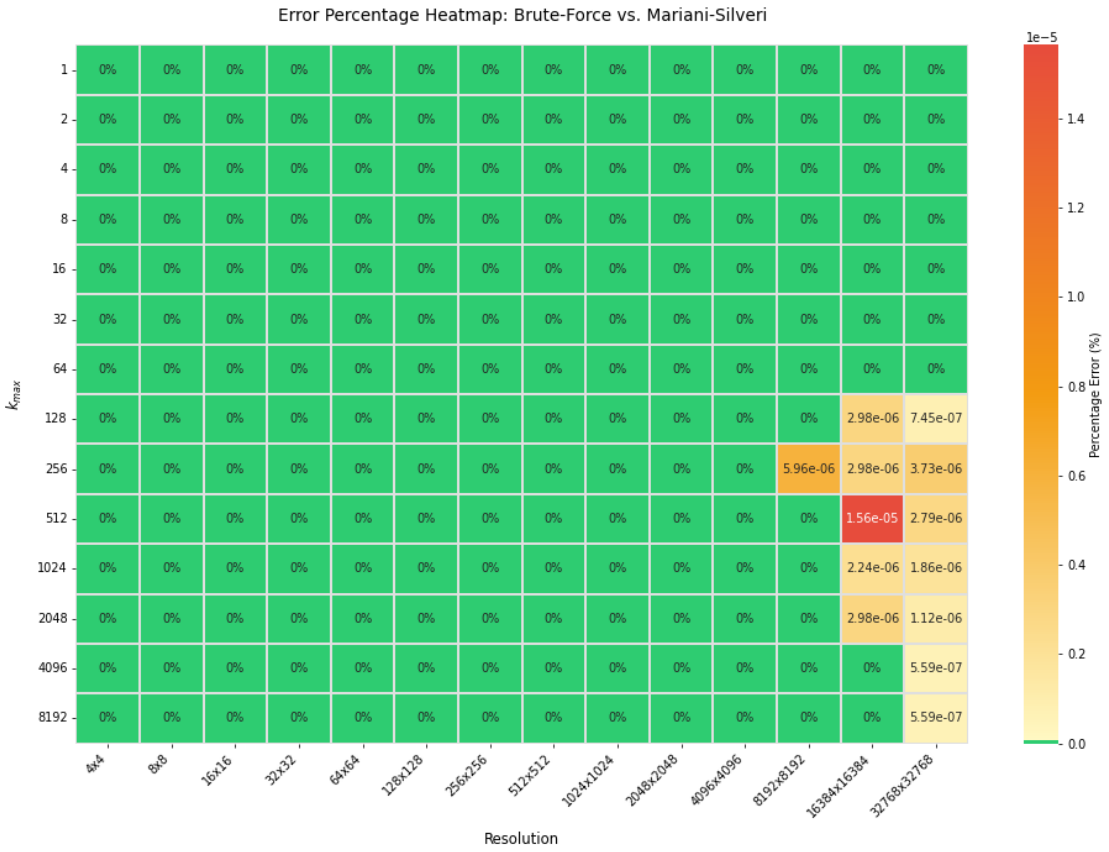

Figure 1: Percentage error heatmap comparing brute-force and Mariani-Silveri outputs across various resolutions and $k_{\max}$ thresholds. Cells marked in green indicate complete mathematical agreement (0%), while yellow cells highlight approximation discrepancies along resolved boundaries.

3 Parameter setup

Optimizing the performance of parallel implementations requires a tuning of runtime parameters.

3.1 OpenMP Loop Scheduling (Executed on Leonardo)

The computational workload associated with rendering the Mandelbrot set is highly non-uniform. The spatial distribution of coordinates belonging to the interior of the set (the main cardioid and its secondary bulbs) requires the maximum iteration count ($k_{\max}$) before the escape condition fails. On the other hand, coordinates situated in the outer regions escape the threshold in few iterations. This difference between density zones creates a load imbalance among processors if loop scheduling is not carefully managed.

To study this load imbalance, benchmarks were performed on Leonardo using three primary OpenMP loop-scheduling strategies: `static`, `dynamic` (with chunk sizes of 1, 2, 4, and 8), and `guided`. The testing focused on both the brute-force (`mandel_brute_omp`) and the optimized Mariani-Silveri (`mandel_mariani_omp`) algorithms under resolutions of 4096×4096 and 8192×8192 with $k_{\max} = 512$.

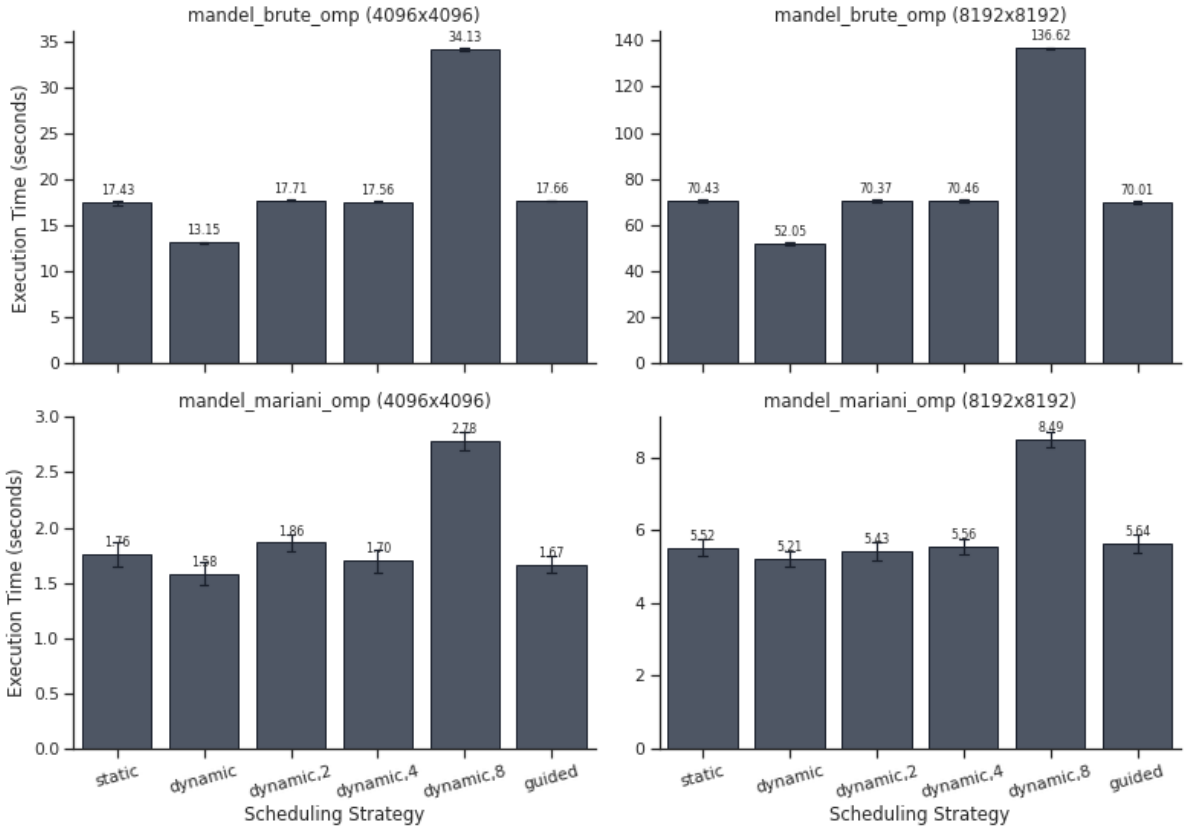

Figure 2: Comparison of average execution times across different OpenMP loop scheduling strategies for the brute-force and Mariani-Silveri implementations on Leonardo.

Results:

- **Static Scheduling:** Using a `static` scheduling, the iteration space is divided into equal, contiguous chunks and pre-allocated to the threads at startup. This approach have a poor execution times ($\approx$ 17.42s for brute-force at 4096×4096 and $\approx$ 70.43 s at 8192×8192). Because some threads are assigned chunks containing dense clusters of inner-set pixels while others process fast-escaping outer pixels, many threads sit idle waiting for the bottleneck processes to complete.

- **Pure Dynamic Scheduling:** The default `dynamic` strategy (chunk=1) had the best execution times across all configurations. For the 8192×8192 brute-force workload, the dynamic strategy reduced the mean runtime to 52.05 s . For the 8192×8192 Mariani-Silveri workload, it achieved a mean runtime of 5.21 s. A chunk size of 1 allows threads to request new rows of pixels immediately upon completing their previous assignment, effectively mitigating load imbalance at the cost of minimal runtime scheduling overhead.
- **Impact of Chunk Size:** As the chunk size for the `dynamic` scheduler is increased to 2, 4, and 8, performance degrades progressively. For example, at a chunk size of 8, the execution time for the 8192×8192 brute-force run scales up to 136.62 s, which is worse than the static one. Larger chunk sizes group adjacent lines of pixels together, that reintroduces the spatial load imbalance characteristic of the static strategy.
- **Guided Scheduling:** The `guided` strategy, which begins with large chunk sizes and dynamically decreases them to handle remaining iterations, achieved execution times that closely mirror the static baseline (≈ 17.66 s at 4096×4096 and ≈ 70.01 s at 8192×8192). Because the initial chunk sizes are too large, they fail to resolve the early-stage load imbalances associated with the complex central cardioid.

Based on these results, the pure `dynamic` scheduling strategy with a chunk size of 1 was selected as the optimal OpenMP configuration for all subsequent scalability evaluations.

3.2 Analysis on `factor` and `brute` parameters (Executed on Orfeo)

The performance of the parallel Mariani-Silveri implementation is highly sensitive to two key parameters: the initial split `factor` and the brute-force threshold (`brute`). Both parameters must be tuned to avoid performance bottlenecks.

3.2.1 The Influence of the `factor` Parameter

The `factor` parameter controls the initial division of the computational domain.

- **Too Small:** If the factor is too small, the initial tiles are too large. Workers spend excessive time recursively splitting these tiles instead of calculating pixels, which increases management overhead.
- **Too Large:** If the factor is too large, the initial tiles are too small and are solved almost instantly. This causes workers to constantly request new work from the master, leading to high communication overhead and network congestion.

The optimal value depends heavily on the target resolution and also on the `brute` parameter. For a high-resolution grid of 32768×32768 and a meaningful `brute` parameter, empirical data suggests that a `factor` between 32 and 64 provides a better balance between task granularity and communication cost.

3.2.2 The Influence of the `brute` Parameter

The `brute` parameter defines the minimum size threshold below which a tile is computed using brute force rather than being split further.

- **Too Small:** If the brute threshold is too small, the algorithm continues to split tasks into very small tiles, generating unnecessary splitting overhead.
- **Too Large:** If the brute threshold is too large, the splitting overhead is reduced, but the algorithm cannot fully exploit the Mariani-Silveri boundary optimization. The workers fallback to brute-force evaluation too early, losing the benefits of the heuristic.

For the tested configurations, the optimal `brute` threshold was found to be around 8192, as it successfully balances the computational savings of the boundary optimization with the cost of recursive splitting.

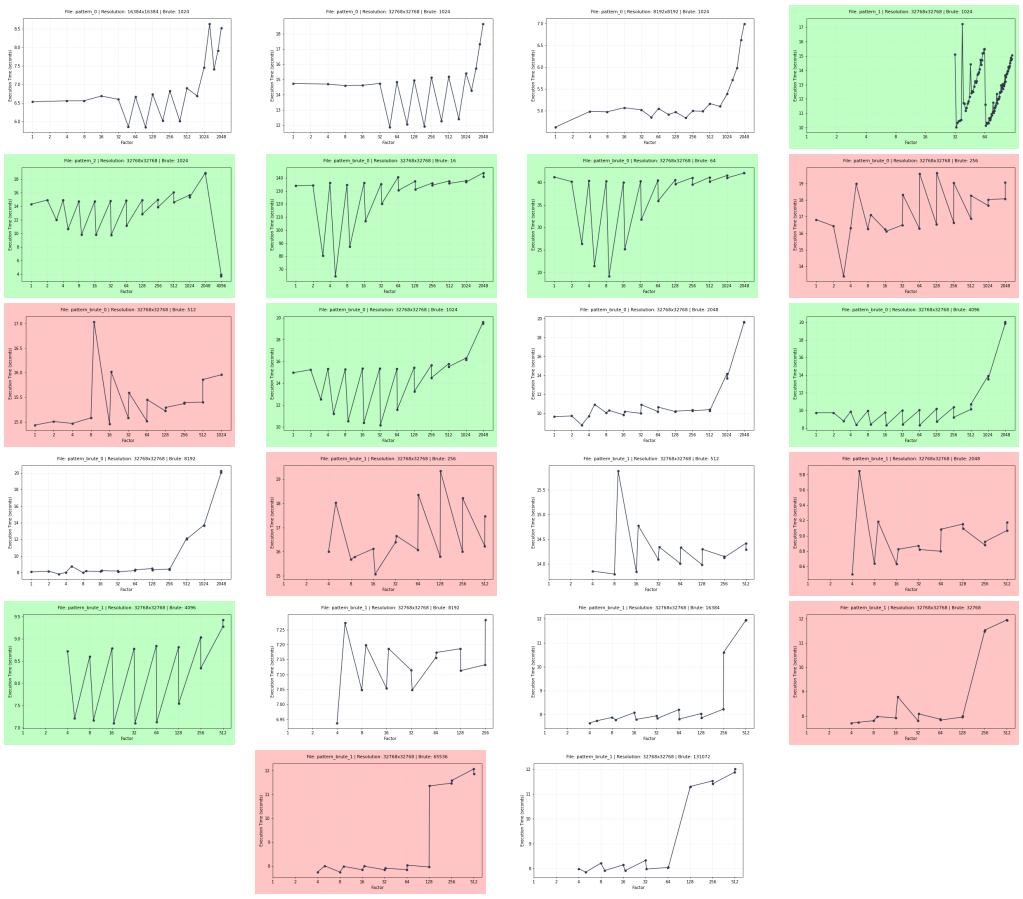

Figure 3: Complete grid of the 22 performance patterns showing the scaling behavior.

4 Scalability Analysis

4.1 Strong Scaling & Load Balance (Executed on Leonardo)

To evaluate the parallel performance and scalability of the master-worker implementation, strong-scaling tests were performed. The problem size was kept constant at a high resolution of 8192×8192 pixels, while the number of total MPI processes (P) was scaled from 2 to 32 across different computational intensities ($k_{\max}$ ranging from 1024 to 16384).

Because our implementation adopts a dedicated master-worker model, the Master process (Rank 0) exclusively coordinates task dispatching, queue management, and image assembly, while only the remaining $P-1$ processes function as active compute workers. Consequently, the ideal linear speedup is bounded by $P-1$, rather than P.

4.1.1 Strong Scaling and Parallel Efficiency

As shown in Figure 4 (left), the speedup curves demonstrate a dependency on the computational intensity of the workload. For low iteration limits ($k_{\max} = 1024$), the execution times of the individual tiles are short, causing the communication overhead.

On the other hand, when increasing the computational intensity to $k_{\max} = 16384$, the ratio of calculation to communication increases significantly.

This behavior is further validated by the parallel efficiency trends in Figure 4 (right). Across all process counts, parallel efficiency increases linearly as $k_{\max}$ increases. For smaller allocations ($P = 4$, i.e., 3 workers), the parallel efficiency is exceptionally high. For larger allocations ($P = 32$), the efficiency starts low at 15% due to task-starvation and communication bottlenecks at low iteration counts, but the efficiency increases at higher computational intensities.

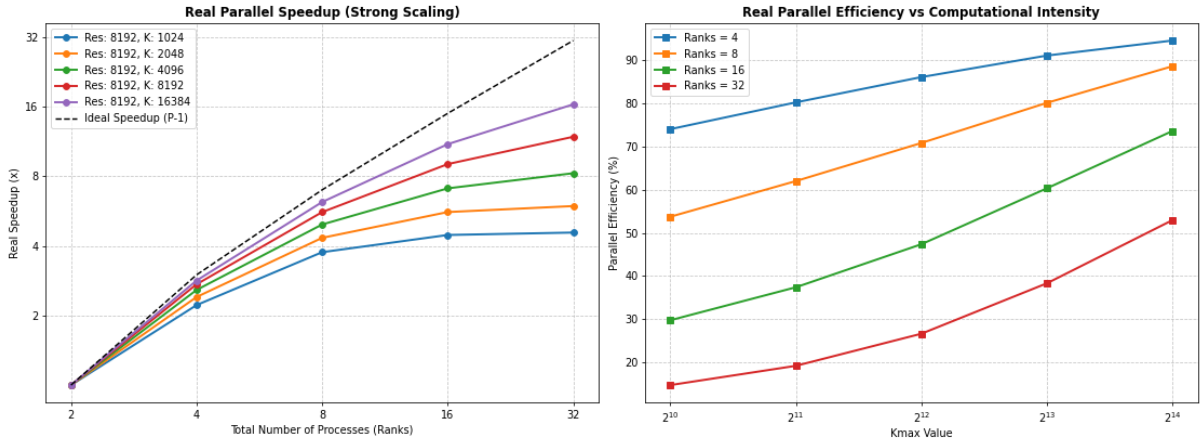

Figure 4: Strong scaling speedup (left) and parallel efficiency as a function of computational intensity $k_{\max}$ (right) for an 8192×8192 grid.

4.1.2 Load Balance Analysis

To evaluate the effectiveness of the dynamic task-assignment protocol, the load imbalance among the computing workers was quantified. The load imbalance metric represents the relative deviation of the workers' computing times from the total average. In other words it is the proportion of time lost to idle waiting.

As shown in Figure 5 (left), the measured load imbalance remains exceptionally low across all configurations, staying well below the strict 10% target limit indicated by the red dashed line.

The Master process maintains an active queue of tile blocks and distributes them reactively to workers as soon as they become idle. Consequently:

- Workers assigned to low-density, fast-escaping outer regions of the Mandelbrot set complete their tiles quickly and immediately request new ones, avoiding idle periods.
- Workers assigned to high-density inner regions of the cardioid compute continuously without stalling other processes, as their slower progress does not block the asynchronous assignment of the remaining queued tiles.

This behavior is confirmed by the median worker execution times shown in Figure 5 (right), which decrease smoothly as the number of processes increases, confirming the robust design of the dynamic parallel loop.

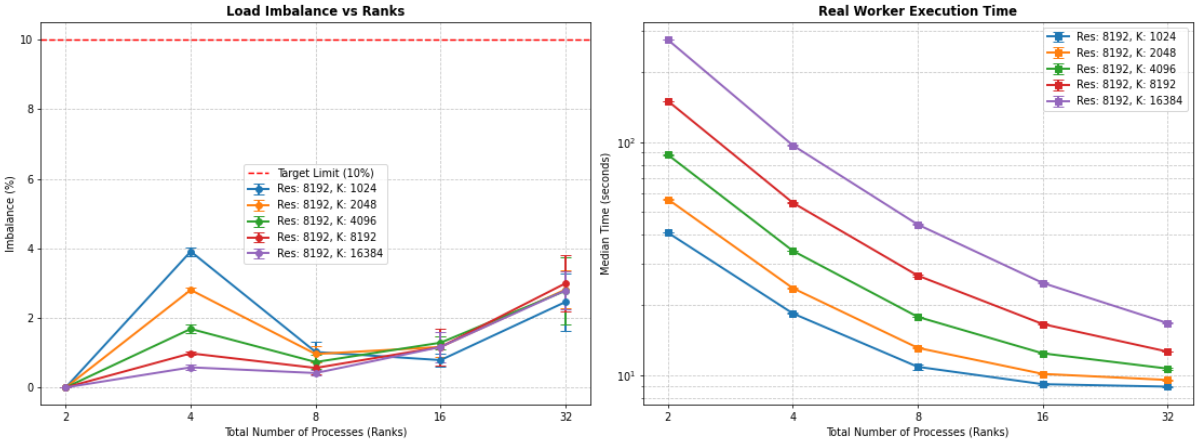

Figure 5: Load imbalance percentage relative to the 10% target limit (left) and median worker execution times on a logarithmic scale (right).

4.2 Weak Scaling Analysis (Executed on Orfeo)

In weak scaling, the workload allocated to each individual compute worker is kept constant, while the total problem size scales proportionally with the number of active workers $(P-1)$. For this benchmark, the base grid resolution was scaled relative to the worker count to maintain a constant computational load per processor. In an ideal parallel system with no communication overhead, the average execution time would remain completely flat, corresponding to a constant parallel efficiency of 100%.

The empirical results of the weak scaling benchmarks performed on Orfeo are presented in Figure 6.

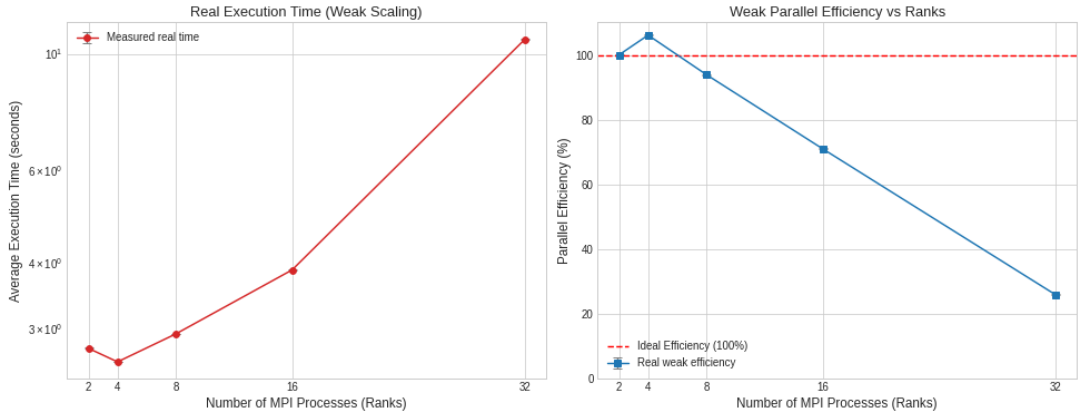

Figure 6: Real execution time (left) and weak parallel efficiency (right) as a function of the total number of MPI processes (Ranks) on Orfeo. The dashed red line represents the ideal weak scaling efficiency (100%).

The weak scaling curves reveal distinct operational phases of the parallel framework:

- **Super-linear Efficiency at Low Process Counts:** At $P = 4$ processes (corresponding to 3 compute workers), the execution time drops slightly to ≈ 2.6 seconds, yielding a parallel efficiency of approximately 106%. This super-linear behavior is a known characteristic of centralized master-worker configurations at low scaling limits. It occurs because the fixed administrative overhead of the Master process (Rank 0) is better amortized across multiple workers.
- **Bottleneck at 32 Ranks:** At $P = 32$ processes (31 active workers), the parallel efficiency drops approximately into 25%. Despite the performance drop at larger scales, the parallel efficiency decreases in a strictly linearly with respect to the number of processes (losing approximately 2.9% of efficiency per added rank across the entire range from 4 to 32 processes). This constant rate of decay indicates that the communication overhead scales linearly rather than exponentially, demonstrating that the system's performance remains predictable.

The weak scaling analysis indicates that while the dynamic master-worker scheme is highly effective at managing load imbalance for moderate process counts, it exhibits a clear scalability limit at higher process counts due to the centralized communication bottleneck at the Master rank.

4.3 Master Process Saturation and Rank 0 Bottleneck (Executed on Orfeo)

Even if the dynamic master-worker paradigm is effective at reducing load imbalances, the single Master process (Rank 0) becomes saturated as the number of processors increases. It is profiled the parallel execution times and quantified the percentage of time workers spent idling.

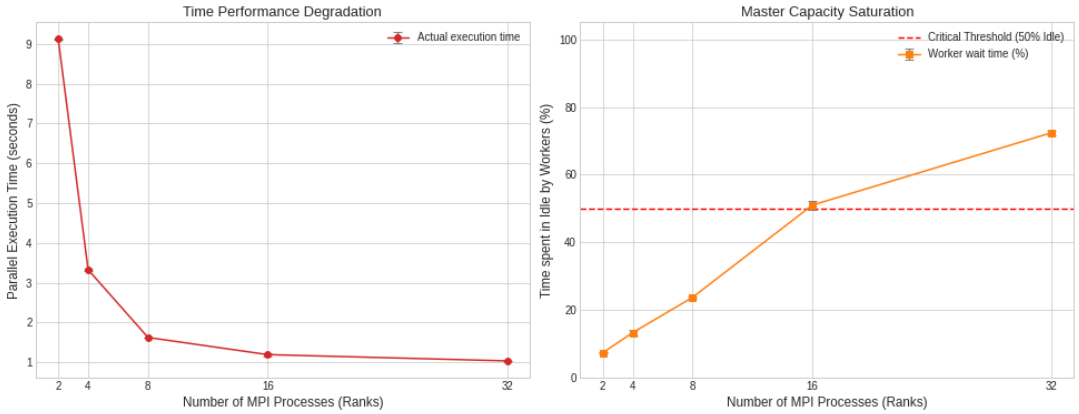

Figure 7: Parallel execution time degradation (left) and worker idle time saturation relative to the 50% critical threshold (right) as a function of the total number of MPI processes (Ranks) on Orfeo.

Results:

- **Execution Time Flatlining:** As shown in Figure 7 (left), the parallel execution time drops sharply from 9.12 s at 2 ranks to 1.61 s at 8 ranks, demonstrating efficient parallel scaling at lower process counts. However, beyond 8 ranks, the performance gains degrade rapidly, asymptotically approaching a hard limit. Increasing the process count from 16 to 32 ranks yields only a negligible performance improvement, with the runtime flatlining at 1.03 s.
- **Worker Idle Time Accumulation:** The explanation for this scaling degradation is provided by the worker idle time profile in Figure 7 (right). The percentage of time spent in an idle state by the computing workers scales almost linearly with the number of allocated processes. At 2 ranks (1 worker), the idle time is low ($\approx 7\%$). This idle fraction increases to 24% at 8 ranks, and at 16 ranks, it reaches 51%, crossing the critical 50% threshold where workers spend more time waiting than performing actual computations.

These findings demonstrate that while the dynamic master-worker scheme successfully maintains load balance, the serial bottleneck at Rank 0 imposes a strict limit on the scalability of the implementation. To scale effectively beyond 16 processes, a decentralized or hierarchical task-distribution model would be required.

5 Performance Comparison: Native vs. Containerized Execution

This section provides a detailed analysis of the performance of the application, comparing the real execution times (t_{real}) and their stability between the native cluster environment and the containerized environment managed via Singularity/Apptainer. The complete benchmark results are presented in Table **??**.

5.1 Analysis of Pure OpenMP Configurations

The purer shared-memory OpenMP configurations (`16_omp` and `64_omp`) exhibit highly competitive results between the two environments, with performance discrepancies generally remaining within a 10% margin.

- **16-Thread Configuration (`16_omp`):** For the smaller workload ($K_{max} = 8192$), performance is nearly identical. However, as the computational load increases , native execution demonstrates a performance advantage. Furthermore, significantly higher variability is observed in the containerized run ($\sigma_{\text{container}} = 1.74\,\text{s}$ versus $\sigma_{\text{native}} = 0.0089\,\text{s}$). This suggests that the containerization layer or its interactions with system calls may be more susceptible to system jitter and OS noise under heavy memory and core utilization.
- **64-Thread Configuration (`64_omp`):** Interestingly, a minor counterintuitive trend is observed: the containerized run is marginally faster both for low workloads and high workloads. This small difference may be attributed to differences in the underlying runtime libraries or a more optimal default thread affinity policy (*thread pinning*) within the isolated environment.

5.2 Analysis of Pure MPI Configurations

- **16-Task Scalability (`16_mpi`):** The native environment shows a systematic but minor performance advantage of approximately across both workloads. This behavior aligns with typical high-performance computing expectations, where the network virtualization or container abstraction layer introduces a minimal, predictable latency penalty during inter-process and collective MPI communications.
- **64-Task Scalability (`64_mpi`):** For the smaller workload ($K_{max} = 8192$), both environments perform comparably.

  However, at the higher workload ($K_{max} = 16384$), a critical performance degradation occurs: the containerized run consistently reaches the 900.00 s threshold, reaching the job timeout or an unresolved deadlock within the container.

5.3 Analysis of Hybrid Configurations (MPI + OpenMP)

- `4_mpi_4_omp` **Configuration:** The container significantly outperforms the native run, yielding a 2.39× speedup for the smaller workload and a 1.60× speedup for the larger workload.
- `8_mpi_8_omp` **Configuration:** Similarly, for small workloads, the container maintains a modest advantage but this time it doesn't oversmart. As the workload scales up, the trend partially reverses in favor of the native environment.

The leading hypothesis for the container's superior performance in hybrid scenarios relates to NUMA domain management and CPU affinity. In modern multi-socket architectures (such as the architecture of the `GENOA` partition), a lack of proper OpenMP thread binding relative to

their parent MPI processes can lead to thread migration across NUMA nodes. This migration introduces high remote-memory access latencies. The Singularity container, in combination with the specific OpenMPI runtime launched, appears to enforce a stricter and more hardware-compliant default process/thread pinning scheme.

5.4 Observations on Jitter and Measurement Stability

The native environment consistently exhibits much lower standard deviations. This behavior demonstrates that native execution, by minimizing additional software abstraction layers and container filesystem mounts, delivers superior performance predictability, which is highly desirable for precision benchmarking and production-level workloads.

Table 3: Performance comparison (Mean ± Standard Deviation in seconds) between Container and Native.

Configuration	K_{max}	Brute	Container (s)	Native (s)	Best	Speedup
4_mpi_4_omp	8192	1024	74.3922 ± 0.5424	177.9880 ± 0.9224	**Container**	2.39x
4_mpi_4_omp	16384	16384	148.9127 ± 0.2537	237.8775 ± 0.3670	**Container**	1.60x
8_mpi_8_omp	8192	1024	40.4429 ± 0.2553	43.4456 ± 0.1121	**Container**	1.07x
8_mpi_8_omp	16384	16384	57.4271 ± 0.3849	55.0560 ± 0.0437	**Native**	1.04x
16_mpi	8192	1024	28.5725 ± 0.1142	27.6935 ± 0.0427	**Native**	1.03x
16_mpi	16384	16384	85.2778 ± 0.2124	82.1569 ± 0.0284	**Native**	1.04x
16_omp	8192	1024	22.7878 ± 0.1902	22.7651 ± 0.0517	**Native**	1.00x
16_omp	16384	16384	44.8013 ± 1.7463	40.3709 ± 0.0089	**Native**	1.11x
64_mpi	8192	1024	18.8507 ± 2.2545	19.0353 ± 0.0600	**Container**	1.01x
64_mpi	16384	16384	900.0000 ± 0.0000	29.2549 ± 0.0736	**Native**	30.76x
64_omp	8192	1024	9.0825 ± 0.0750	9.3051 ± 0.0082	**Container**	1.02x
64_omp	16384	16384	15.7851 ± 0.0761	16.4667 ± 0.0087	**Container**	1.04x